09-02 메트릭 및 모니터링 시스템 분석

개요

Solana의 메트릭 및 모니터링 시스템은 네트워크의 성능, 상태, 그리고 동작을 실시간으로 추적하고 분석할 수 있는 종합적인 도구를 제공합니다. 이 시스템은 InfluxDB를 백엔드로 사용하여 시계열 데이터를 수집하고 저장합니다.

1. 메트릭 시스템 아키텍처

1.1 전체 구조

```
┌─────────────────┐  ┌──────────────┐  ┌──────────────┐
│ Application │──▶│ MetricsAgent │──▶│ InfluxDB │
│ Components      │ │              │ │   Backend   │
└─────────────────┘  └──────────────┘  └──────────────┘
│ │
▼ ▼
┌─────────────────┐  ┌──────────────┐
│ DataPoint │ │ Counter │
│ Macros │ │ System │
└─────────────────┘  └──────────────┘
```

1.2 핵심 컴포넌트

1. **MetricsAgent**: 메트릭 수집 및 전송 관리
2. **DataPoint**: 개별 메트릭 데이터 포인트
3. **Counter**: 누적 카운터 시스템
4. **InfluxDbMetricsWriter**: InfluxDB 백엔드 연동

2. DataPoint 시스템

2.1 DataPoint 구조체

```rust
#[derive(Clone, Debug)]
pub struct DataPoint {
pub name: &'static str,
pub timestamp: SystemTime,
/// tags are eligible for group-by operations.
pub tags: Vec<(&'static str, String)>,
pub fields: Vec<(&'static str, String)>,
}
```

주요 구성 요소:

- `name` : 메트릭 이름 (정적 문자열)
- `timestamp` : 측정 시점
- `tags` : 그룹화 가능한 태그들 (인덱싱 용도)
- `fields` : 실제 측정값들

2.2 DataPoint 메서드

기본 생성

```
pub fn new(name: &'static str) -> Self {
DataPoint {
name,
timestamp: SystemTime::now(),
tags: vec![],
fields: vec![],
}
}
```

필드 추가 메서드

```
pub fn add_field_str(&mut self, name: &'static str, value: &str) -> &mut Sel
self.fields
.push((name, format!("\"{}\"", value.replace('\"', "\\\""))));
self
}

pub fn add_field_bool(&mut self, name: &'static str, value: bool) -> &mut Se
self.fields.push((name, value.to_string()));
self
}
```

```
pub fn add_field_i64(&mut self, name: &'static str, value: i64) -> &mut Self
self.fields.push((name, value.to_string() + "i"));
self
}

pub fn add_field_f64(&mut self, name: &'static str, value: f64) -> &mut Self
self.fields.push((name, value.to_string()));
self
}
```

태그 추가

```
pub fn add_tag(&mut self, name: &'static str, value: &str) -> &mut Self {
self.tags.push((name, value.to_string()));
self
}
```

2.3 DataPoint 매크로 시스템

기본 매크로 구조

```
#[macro_export]
macro_rules! create_datapoint {
(@field $point:ident $name:expr, $string:expr, String) => {
```

```
$point.add_field_str($name, &$string);
};
(@field $point:ident $name:expr, $value:expr, i64) => {
$point.add_field_i64($name, $value  as i64);
};
(@field $point:ident $name:expr, $value:expr, f64) => {
$point.add_field_f64($name, $value  as f64);
};
(@field $point:ident $name:expr, $value:expr, bool) => {
$point.add_field_bool($name, $value  as bool);
};
// ... 기타 매크로 규칙들
}
```

로그 레벨별 매크로

```
#[macro_export]
macro_rules!  datapoint_error {
($name:expr, $($fields:tt)+) => {
$crate::datapoint!(log::Level::Error, $name, $($fields)+);
};
}

#[macro_export]
```

```
macro_rules!  datapoint_warn {
($name:expr, $($fields:tt)+) => {
$crate::datapoint!(log::Level::Warn, $name, $($fields)+);
};
}

#[macro_export]
macro_rules!  datapoint_info {
($name:expr, $($fields:tt)+) => {
$crate::datapoint!(log::Level::Info, $name, $($fields)+);
};
}

#[macro_export]
macro_rules!  datapoint_debug {
($name:expr, $($fields:tt)+) => {
$crate::datapoint!(log::Level::Debug, $name, $($fields)+);
};
}
```

사용 예시

```
datapoint_info!(
```

```
"validator_performance",
"node_id" => "validator_123",
"cluster" => "mainnet-beta",
("tps", 50000, i64),
("cpu_usage", 75.5, f64),
("memory_usage", 8192, i64),
("is_leader", true, bool),
);
```

3. MetricsAgent 시스템

3.1 MetricsAgent 구조체

```
pub  struct MetricsAgent {
sender: Sender<MetricsCommand>,
}
```

3.2 MetricsCommand 열거형

```
#[derive(Debug)]
enum MetricsCommand {
Flush(Arc<Barrier>),
Submit(DataPoint, log::Level),
SubmitCounter(CounterPoint, log::Level, u64),
```

```
}
```

명령 유형:

- `Flush` : 즉시 모든 대기 중인 메트릭 전송
- `Submit` : DataPoint 제출
- `SubmitCounter` : Counter 제출

3.3 MetricsAgent 생성 및 실행

기본 생성자

```
impl Default for MetricsAgent {
fn  default() -> Self {
let  max_points_per_sec = env::var("SOLANA_METRICS_MAX_POINTS_PER_SECOND")
.map(|x| {
x.parse()
.expect("Failed to parse SOLANA_METRICS_MAX_POINTS_PER_SECOND")
})
.unwrap_or(4000);

Self::new(
Arc::new(InfluxDbMetricsWriter::new()),
Duration::from_secs(10),
max_points_per_sec,
)
```

```
}
}
```

메인 실행 루프

```
fn run(
receiver: &Receiver<MetricsCommand>,
writer: &Arc<dyn MetricsWriter + Send + Sync>,
write_frequency: Duration,
max_points_per_sec: usize,
) {
let mut last_write_time = Instant::now();
let mut points = Vec::<DataPoint>::new();
let mut counters = CounterMap::new();
let max_points = write_frequency.as_secs() as usize * max_points_per_sec;

loop {
match receiver.recv_timeout(write_frequency / 2) {
Ok(cmd) => match cmd {
MetricsCommand::Flush(barrier) => {
Self::write(
writer,
Self::collect_points(&mut points, &mut counters),
```

```
max_points,
max_points_per_sec,
last_write_time,
receiver.len(),
);
last_write_time = Instant::now();
barrier.wait();
}
MetricsCommand::Submit(point, level) => {
log!(level, "{}", point);
points.push(point);
}
MetricsCommand::SubmitCounter(counter, _level, bucket) => {
let  key = (counter.name, bucket);
if  let Some(value) = counters.get_mut(&key) {
value.count += counter.count;
} else {
counters.insert(key, counter);
}
}
},
Err(RecvTimeoutError::Timeout) => {
// 주기적 전송 체크
```

```
}
Err(RecvTimeoutError::Disconnected) => {
break;
}
}

// 주기적 전송
let  now = Instant::now();
if  now.duration_since(last_write_time) >= write_frequency {
Self::write(
writer,
Self::collect_points(&mut  points, &mut  counters),
max_points,
max_points_per_sec,
last_write_time,
receiver.len(),
);
last_write_time = now;
}
}
}
```

3.4 속도 제한 및 최적화

최대 포인트 제한

```
fn write(
writer: &Arc<dyn MetricsWriter + Send + Sync>,
mut points: Vec<DataPoint>,
max_points: usize,
max_points_per_sec: usize,
last_write_time: Instant,
points_buffered: usize,
) {
if points.is_empty() {
return;
}


let num_points = points.len();
if num_points > max_points {
warn!(
"max submission rate of {} datapoints per second exceeded. only the
first {} of {} points will be submitted",
max_points_per_sec, max_points, num_points
);
}
let points_written = cmp::min(num_points, max_points - 1);
```

```
points.truncate(points_written);

// 메타 메트릭 추가

points.push(

DataPoint::new("metrics")

.add_field_i64("points_written", points_written  as i64)

.add_field_i64("num_points", num_points  as i64)

.add_field_i64("points_lost", (num_points - points_written) as i64)

.add_field_i64("points_buffered", points_buffered  as i64)

.add_field_i64(

"secs_since_last_write",

now.duration_since(last_write_time).as_secs() as i64,

)

.to_owned(),

);



writer.write(points);

}
```

4. Counter 시스템

4.1 Counter 구조체

```
pub  struct Counter {
```

```
pub name: &'static str,
/// total accumulated value
pub counts: AtomicUsize,
pub times: AtomicUsize,
/// last accumulated value logged
pub lastlog: AtomicUsize,
pub lograte: AtomicUsize,
pub metricsrate: AtomicU64,
}
```

4.2 CounterPoint 구조체

```
#[derive(Clone, Debug)]
pub struct CounterPoint {
pub name: &'static str,
pub count: i64,
pub timestamp: SystemTime,
}
```

4.3 Counter 매크로 시스템

Counter 생성 매크로

```
#[macro_export]
```

```
macro_rules!  create_counter {
($name:expr, $lograte:expr, $metricsrate:expr) => {
$crate::counter::Counter {
name: $name,
counts: std::sync::atomic::AtomicUsize::new(0),
times: std::sync::atomic::AtomicUsize::new(0),
lastlog: std::sync::atomic::AtomicUsize::new(0),
lograte: std::sync::atomic::AtomicUsize::new($lograte),
metricsrate: std::sync::atomic::AtomicU64::new($metricsrate),
}
};
}
```

Counter 증가 매크로

```
#[macro_export]
macro_rules!  inc_new_counter_info {
($name:expr, $count:expr) => {{
inc_new_counter!($name, $count, log::Level::Info, 0, 0);
}};
($name:expr, $count:expr, $lograte:expr) => {{
inc_new_counter!($name, $count, log::Level::Info, $lograte, 0);
}};
($name:expr, $count:expr, $lograte:expr, $metricsrate:expr) => {{
```

```
inc_new_counter!($name, $count, log::Level::Info, $lograte, $metricsrate);
}};
}
```

4.4 Counter 증가 로직

```
pub fn inc(&mut self, level: log::Level, events: usize) {
let now = timing::timestamp();
let counts = self.counts.fetch_add(events, Ordering::Relaxed);
let times = self.times.fetch_add(1, Ordering::Relaxed);
let lograte = self.lograte.load(Ordering::Relaxed);
let metricsrate = self.metricsrate.load(Ordering::Relaxed);

// 로그 출력 (주기적)
if times % lograte == 0 && times > 0 && log_enabled!(level) {
log!(level,
"COUNTER:{{\"name\": \"{}\", \"counts\": {}, \"samples\": {}, \"now\": {}, \"eve
self.name,
counts + events,
times,
now,
events,
);
```

```
}

// 메트릭 제출 (버킷 기반)
let lastlog = self.lastlog.load(Ordering::Relaxed);
let prev = self
.lastlog
.compare_and_swap(lastlog, counts, Ordering::Relaxed);
if prev == lastlog {
let bucket = now / metricsrate;
let counter = CounterPoint {
name: self.name,
count: counts as i64 - lastlog as i64,
timestamp: SystemTime::now(),
};
submit_counter(counter, level, bucket);
}
}
```

5. InfluxDB 백엔드 시스템

5.1 InfluxDbMetricsWriter

```
struct InfluxDbMetricsWriter {
```

```
write_url: Option<String>,
}
```

5.2 설정 관리

MetricsConfig 구조체

```
#[derive(Debug, Default)]
struct MetricsConfig {
pub host: String,
pub db: String,
pub username: String,
pub password: String,
}
```

환경 변수 파싱

```
fn get_metrics_config() -> Result<MetricsConfig, MetricsError> {
let mut config = MetricsConfig::default();
let config_var = env::var("SOLANA_METRICS_CONFIG")?;
for pair in config_var.split(',') {
let nv: Vec<_> = pair.split('=').collect();
if nv.len() != 2 {
return Err(MetricsError::ConfigInvalid(pair.to_string()));
```

```
}
let v = nv[1].to_string();
match nv[0] {
"host" => config.host = v,
"db" => config.db = v,
"u" => config.username = v,
"p" => config.password = v,
_ => return Err(MetricsError::ConfigInvalid(pair.to_string())),
}
}

if !config.complete() {
return Err(MetricsError::ConfigIncomplete);
}

Ok(config)
}
```

5.3 데이터 직렬화

InfluxDB Line Protocol 생성

```
pub fn serialize_points(points: &Vec<DataPoint>, host_id: &str) -> String {
let mut line = String::with_capacity(len);
```

```
for point in points {
let _ = write!(line, "{},host_id={}", &point.name, host_id);
// 태그 추가
for (name, value) in point.tags.iter() {
let _ = write!(line, ",{name}={value}");
}

// 필드 추가
let mut first = true;
for (name, value) in point.fields.iter() {
let _ = write!(line, "{}{}={}", if first { ' ' } else { ',' }, name, value);
first = false;
}
// 타임스탬프 추가
let timestamp = point.timestamp.duration_since(UNIX_EPOCH);
let nanos = timestamp.unwrap().as_nanos();
let _ = writeln!(line, " {nanos}");
}
line
}
```

5.4 HTTP 전송

```
impl MetricsWriter for InfluxDbMetricsWriter {

fn  write(&self, points: Vec<DataPoint>) {

if  let Some(ref  write_url) = self.write_url {

let  host_id = HOST_ID.read().unwrap();

let  line = serialize_points(&points, &host_id);



let  client = reqwest::blocking::Client::builder()

.timeout(Duration::from_secs(5))

.build();

let  response = client.post(write_url.as_str()).body(line).send();

if  let Ok(resp) = response {

let  status = resp.status();

if !status.is_success() {

let  text = resp

.text()

.unwrap_or_else(|_| "[text body empty]".to_string());

warn!("submit response unsuccessful: {} {}", status, text,);

}

} else {

warn!("submit error: {}", response.unwrap_err());

}

}
```

```
}

}
```

6. 고급 모니터링 기능

6.1 MovingStat - 외부 카운터 추적

```
#[derive(Default)]

pub struct MovingStat {

value: AtomicU64,

}


impl MovingStat {

pub fn update_stat(&self, old_value: &MovingStat, new_value: u64) {

let old = old_value.value.swap(new_value, Ordering::Acquire);

self.value

.fetch_add(new_value.saturating_sub(old), Ordering::Release);

}


pub fn load_and_reset(&self) -> u64 {

self.value.swap(0, Ordering::Acquire)

}

}
```

6.2 TokenCounter - 리소스 추적

```
pub struct TokenCounter(Arc<&'static str>);

impl TokenCounter {
pub fn new(name: &'static str) -> Self {
Self(Arc::new(name))
}

pub fn create_token(&self) -> CounterToken {
// Arc 참조 카운트를 이용한 자동 추적
datapoint_info!(*self.0, ("count", Arc::strong_count(&self.0), i64));
CounterToken(self.0.clone())
}
}

pub struct CounterToken(Arc<&'static str>);

impl Drop for CounterToken {
fn drop(&mut self) {
// 토큰 소멸 시 자동으로 카운트 감소 보고
datapoint_info!(
*self.0,
```

```
("count", Arc::strong_count(&self.0).saturating_sub(2), i64)
);
}
}
```

6.3 패닉 후킹 시스템

```
pub fn set_panic_hook(program: &'static str, version: Option<String>) {
static SET_HOOK: Once = Once::new();
SET_HOOK.call_once(|| {
let default_hook = std::panic::take_hook();
std::panic::set_hook(Box::new(move |ono| {
default_hook(ono);
let location = match ono.location() {
Some(location) => location.to_string(),
None => "?".to_string(),
};
submit(
DataPoint::new("panic")
.add_field_str("program", program)
.add_field_str("thread", thread::current().name().unwrap_or("?"))
.add_field_i64("one", 1)
.add_field_str("message", &ono.to_string())
```

```
.add_field_str("location", &location)
.add_field_str("version", version.as_ref().unwrap_or(&"".to_string()))
.to_owned(),
Level::Error,
);
flush();
std::process::exit(1);
}));
});
}
```

7. 클러스터별 메트릭 검증

7.1 클러스터 타입 검증

```
pub fn metrics_config_sanity_check(cluster_type: ClusterType) -> Result<(), Me
let config = match get_metrics_config() {
Ok(config) => config,
Err(MetricsError::VarError(env::VarError::NotPresent)) => return Ok(()),
Err(e) => return Err(e),
};
match &config.db[..] {
"mainnet-beta" if cluster_type != ClusterType::MainnetBeta => (),
```

```
"tds"  if  cluster_type != ClusterType::Testnet => (),

"devnet"  if  cluster_type != ClusterType::Devnet => (),

_ => return Ok(()),

};

let (host, db) = (&config.host, &config.db);

let  msg = format!("cluster_type={cluster_type:?} host={host} database={db}");

Err(MetricsError::DbMismatch(msg))

}
```

8. 실제 사용 예시

8.1 성능 메트릭 보고

```
// TPU 성능 메트릭

datapoint_info!(

"tpu_performance",

"stage" => "banking",

("transactions_processed", tx_count, i64),

("processing_time_ms", duration_ms, i64),

("queue_length", queue_len, i64),

("cpu_usage_percent", cpu_usage, f64),

);
```

```
// 네트워크 메트릭
datapoint_warn!(
"network_issues",
"peer_id" => peer_id.to_string(),
("packet_loss_rate", loss_rate, f64),
("connection_errors", error_count, i64),
("retry_attempts", retry_count, i64),
);
```

8.2 Counter 사용

```
// 트랜잭션 카운터
inc_new_counter_info!("transactions_received", 1);
inc_new_counter_info!("transactions_processed", batch_size);
inc_new_counter_error!("transaction_errors", 1, 100); // 100번마다 로그
```

8.3 리소스 추적

```
// 연결 추적
let  connection_counter = TokenCounter::new("active_connections");
let  _connection_token = connection_counter.create_token();
// 연결이 끝나면 자동으로 카운트 감소
```

9. 모니터링 대시보드 연동

9.1 InfluxDB 쿼리 지원

```rust
pub fn query(q: &str) -> Result<String, MetricsError> {
let config = get_metrics_config()?;
let query_url = format!(
"{}/query?u={}&p={}&q={}",
&config.host, &config.username, &config.password, &q
);

let response = reqwest::blocking::get(query_url.as_str())?.text()?;
Ok(response)
}
```

9.2 일반적인 쿼리 패턴

```sql
-- TPS 추이
SELECT mean("transactions_processed")
FROM "tpu_performance"
WHERE time > now() - 1h
GROUP BY time(1m)

-- 오류율 모니터링
```

```
SELECT  sum("transaction_errors") / sum("transactions_received") * 100
FROM  "metrics"
WHERE  time > now() - 5m


-- 노드별 성능 비교
SELECT mean("cpu_usage_percent")
FROM  "validator_performance"
WHERE  time > now() - 1h
GROUP BY  "node_id", time(5m)
```

결론

Solana의 메트릭 및 모니터링 시스템은 다음과 같은 특징을 가집니다:

1. **확장 가능한 아키텍처**: 비동기 메시지 패싱을 통한 고성능 메트릭 수집
2. **유연한 데이터 모델**: 태그와 필드를 통한 다차원 메트릭 지원
3. **자동화된 리소스 추적**: Arc 기반 자동 카운팅 시스템
4. **강력한 매크로 시스템**: 타입 안전하고 사용하기 쉬운 메트릭 API
5. **클러스터 인식**: 네트워크별 메트릭 분리 및 검증
6. **실시간 모니터링**: InfluxDB 연동을 통한 실시간 대시보드 지원
7. **오류 추적**: 패닉 후킹을 통한 자동 오류 보고

이러한 시스템을 통해 Solana는 네트워크의 모든 측면을 실시간으로 모니터링하고, 성능 문제를 신속하게 감지하며, 데이터 기반의 최적화 결정을 내릴 수 있습니다.